

42 Philosophers

Felsefe, varlık, bilgi, ahlak, akıl ve dil gibi temel konular üzerine düşünmeyi ve bunları analiz etmeyi amaçlayan bir disiplindir. Terimin Pythagoras tarafından ortaya atıldığı düşünülmektedir. Felsefi yöntemler arasında sorgulama, eleştirel tartışma ve mantıklı argümanlar yer alır.

Tarihsel olarak, felsefe her türlü bilgi alanını kapsarken, 19. yüzyıldan itibaren psikoloji, sosyoloji ve ekonomi gibi bazı disiplinler ayrılarak bağımsız bilim dalları haline gelmiştir. Ancak sanat, bilim, siyaset gibi alanlarla bağlantılı bazı konular felsefenin içinde kalmaya devam etmiştir.

Felsefenin alt dalları arasında metafizik (gerçekliğin doğası), epistemoloji (bilginin doğası ve sınırları), etik, estetik, siyaset felsefesi, mantık ve bilim felsefesi yer almaktadır.

Thread Nedir ?

Thread, bir programın (process'in) içinde çalışan en küçük yürütme birimidir. **Bir işlem (process) en az bir thread içerir** ve her process içinde birden fazla thread çalışabilir. **Thread'ler aynı bellek alanını paylaşır** bu sayede aynı veri üzerinde işlem yapabilirler. Thread'ler sayesinde programlar **daha hızlı ve verimli** çalışabilir, çünkü işlemler **aynı anda (eşzamanlı) yürütülebilir**.

MultiThreading Nedir ?

Multithreading, bir programın içinde **birden fazla thread'in aynı anda çalıştırılmasıdır**.

Avantajları

Daha hızlı çalışır. Çünkü işlemler aynı anda yürütülür.

Kaynakları verimli kullanır. Birden fazla iş tek bir process içinde paylaşılır.

Daha iyi yanıt süresi sağlar. Kullanıcı etkileşimi olan uygulamalarda (örneğin oyunlar, tarayıcılar) işlemler donmadan devam eder.

▼ 💡 Örnek:

- Web tarayıcılarında **bir sekme donarsa** diğer sekmeler çalışmaya devam eder.
- Oyunlarda **grafikler render edilirken** arka planda ses çalmaya devam eder.

Örnek bir multithreading kullanımı.

```
#include "philo.h"

void *print_numbers() {
    for (int i = 0; i < 5; i++) {
        printf("Thread 1: %d\n", i);
    }
    return NULL;
}
```

```

}

void *print_letters() {
    for (char c = 'A'; c <= 'E'; c++) {
        printf("Thread 2: %c\n", c);
    }
    return NULL;
}

int main(int argc, char **argv)
{
    (void)argc;
    (void)**argv;
    pthread_t thread_1;
    pthread_t thread_2;

    pthread_create(&thread_1, NULL, print_numbers, NULL);
    pthread_create(&thread_2, NULL, print_letters, NULL);
    pthread_join(thread_1, NULL);
    pthread_join(thread_2, NULL);
    return (0);
}

```

Burada thread kullanmamızın sebebi sayılar ve harfler rastgele sırayla yazdırılır çünkü aynı anda çalışıyorlar. Program daha hızlı tamamlanır çünkü işlemler aynı anda gerçekleşir. İşlemler birbirinden bağımsız çalışır.

Process (işlem) nedir ?

Process, bir programın çalıştırılmış hali ve kendi belleğini (RAM) yöneten bir yapıdır.

İşletim sistemi her bir process'e ayrı bir bellek tahsis eder.

Process'ler birbirlerinden bağımsızdır ve biri çökerse diğeri bundan etkilenmez.

Thread ve Process Arasındaki Farklar

Hem **process (süreç)** hem de **thread (iş parçacığı)**, programların çalıştırılması için kullanılan yapılardır. Ancak, **kapsamları, çalışma şekilleri ve bellek kullanımları farklıdır.**

1 Process Nedir?

Bir **process**, bağımsız bir programın çalıştırılmasıyla oluşan bir işletim sistemi birimidir.

 Örneğin:

- **firefox** çalıştırıldığında bir **process** oluşturulur.
- **gedit** açıldığında bir **başka process** başlatılır.

Process Özellikleri:

- Her **process** kendi belleğini (memory space) kullanır.
- Bir process **başka bir process'in** hafızasına erişemez.
- **Process'ler arasında veri paylaşımı zordur**, genellikle **IPC (Inter-Process Communication)** mekanizmaları gerekir:
- **Pipes (Boru Hatları)**
- **Message Queues (Mesaj Kuyrukları)**
- **Shared Memory (Paylaşılan Bellek)**

Process'ler **bağımsız çalışır**, biri çökerse diğerini etkilemez.

Process oluşturmak (fork), thread oluşturmaktan daha **ağır ve yavaştır**, çünkü her process **ayrı bir bellek alanı** tahsis eder.

2 Thread Nedir?

Thread, bir **process** içinde çalışan daha küçük iş parçacıklarıdır.

📌 Örneğin:

- **Google Chrome** çalıştığında bir **process** oluşturur, ancak **her sekme bir thread olabilir**.
- **VS Code** çalıştığında bir **process** başlatır, ancak **dosya açma, yazı yazma, uzantılar farklı thread'ler olabilir**.

◆ Thread Özellikleri:

- Aynı process içinde çalışan birden fazla thread aynı belleği paylaşır.
- **Thread'ler daha hızlıdır**, çünkü yeni bir bellek alanı tahsis edilmez.
- **Thread'ler arasında iletişim kolaydır**, çünkü aynı **heap** ve **data** segmentlerini paylaşırlar.
- Eğer bir thread çökerse, **diğer thread'leri ve tüm process'i etkileyebilir**.
- **Thread oluşturmak (pthread_create)**, process oluşturmaktan **çok daha hızlı ve hafiftir**.

Thread Kullanım Örneği

```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

int x = 2;

void* routine() {
    x += 5;
    sleep(2);
    printf("Value of x: %d\n", x);
}

void* routine2() {
    sleep(2);
    printf("Value of x: %d\n", x);
}
```

```

int main(int argc, char* argv[]) {
    pthread_t t1, t2;
    if (pthread_create(&t1, NULL, &routine, NULL)) {
        return 1;
    }
    if (pthread_create(&t2, NULL, &routine2, NULL)) {
        return 2;
    }
    if (pthread_join(t1, NULL)) {
        return 3;
    }
    if (pthread_join(t2, NULL)) {
        return 3;
    }
    return 0;
}

```

Process Kullanım Örneği

```

#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>

int main(int argc, char* argv[]) {
    int x = 2;
    int pid = fork();
    if (pid == -1) {
        return 1;
    }

    if (pid == 0) {
        x++;
    }
    sleep(2);
    printf("Value of x: %d\n", x);
    if (pid != 0) {
        wait(NULL);
    }
    return 0;
}

```

Philosophers Projesinde Thread Kullanımı Neden Önemli?

Philosophers projesinde her filozof **bağımsız bir şekilde düşünmeli ve yemek yemelidir**. Eğer thread kullanmazsak:

- **Her filozof sırayla çalışır** ve biri yemek yerken diğerleri bekler.
- Programın tamamlanması **çok uzun sürer**.
- **Gerçek dünya senaryosuna** uygun olmaz.

Ancak **thread** kullanırsak:

- Her filozof aynı anda düşünür ve yemek yer.
- Daha gerçekçi bir simülasyon olur.
- Program çok daha hızlı tamamlanır.

Threadler arası veri paylaşımı ve Race Conditions Nedir ?

Thread'ler, aynı süreç içinde paralel olarak çalıştıkları için, genellikle ortak verilere erişim sağlarlar. Ancak bu, veri paylaşımı ile ilgili bazı problemleri de beraberinde getirebilir, bunlardan biri de **race condition** (yarış durumu) durumudur.

Threadler Arası Veri Paylaşımı:

Thread'ler, aynı adres alanında çalıştıkları için birbirlerinin verilerine erişebilirler. Bu, threadler arasında veri paylaşımı sağlayan bazı yöntemlerle yapılabilir:

- **Global Değişkenler:** Birçok thread, aynı global değişkenlere erişebilir. Ancak bu durumda, çoklu thread'ler aynı değişkene eşzamanlı olarak erişirse, veri tutarsızlıkları ve hatalar meydana gelebilir.
- **Statik Değişkenler:** Aynı şekilde, statik değişkenler de birden fazla thread tarafından paylaşılabilir. Ancak burada da dikkat edilmesi gereken noktalar vardır.
- **Shared Memory (Paylaşılan Bellek):** Thread'ler, paylaşılan belleği kullanarak veri paylaşabilirler.
- **Thread Argümanları:** Her thread, başlatılırken belirli argümanlarla başlayabilir ve bu argümanlar thread'ler arasında veri paylaşımı için kullanılabilir.

Race Condition (Yarış Durumu):

Yarış durumu, iki veya daha fazla thread'in aynı veriye eşzamanlı olarak erişmeye çalışması durumunda ortaya çıkar. Bu, verinin tutarsız olmasına veya beklenmeyen davranışlara yol açabilir. Yarış durumu genellikle aşağıdaki durumlardan kaynaklanır:

- **Birden fazla thread, aynı veriyi değiştiriyor:** Eğer bir thread, paylaşılan bir veriyi okurken diğer thread aynı veriyi değiştirirse, veri tutarsızlıkları meydana gelir.
- **Eşzamanlı Yazma:** İki thread, aynı veriye yazma yaparsa, bir thread'in yazdığı veri kaybolabilir ve veri yanlış olabilir.

Race Condition durumuna bir örnek.

```
#include "philos.h"

int shared_counter = 0;

void* increment_counter(void* arg)
{
    (void)arg;
    for (int i = 0; i < 100000; i++) {
        shared_counter++;
    }
    return NULL;
}
```

```

int main(int argc, char **argv)
{
    (void)argv;
    (void)argv;
    pthread_t thread1, thread2;

    pthread_create(&thread1, NULL, increment_counter, NULL);
    pthread_create(&thread2, NULL, increment_counter, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Final counter value: %d\n", shared_counter);
    return 0;
}

```

Bu örnekte, iki thread aynı `shared_counter` değişkenini artırıyor. Ancak her iki thread de aynı anda bu değeri artırmaya çalıştığı için **race condition** meydana gelir. Bu, **shared_counter'in** beklenenden farklı bir değere ulaşmasına sebep olabilir.

Yarış Durumu Örneği Açıklaması:

- **thread1** ve **thread2**, **shared_counter'i** artırmaya çalışıyorlar.
- Ancak, her iki thread de aynı anda **shared_counter'i** okur ve sonra aynı değeri üzerine eklerler.
- Bu durumda, iki thread'in aynı anda **shared_counter'i** artırması sonucunda, artışlar kaybolur. Örneğin, her iki thread de **shared_counter'in** 0 olduğunu görebilir ve her ikisi de 1 ekler, ancak bu durumda sonuç sadece 1 olur. Oysa ki iki thread de 1000000 kez artırma yapacağı için sonuç aslında 2000000 olmalıydı.

Şimdi gelelim Race Condition durumundan korunma yöntemlerine.

Mutex (Mutual Exclusion) !

Mutex (Mutual Exclusion), çoklu iş parçacıkları (thread) veya süreçlerin (process) paylaşılan kaynaklara erişimini senkronize etmek için kullanılan bir yapıdır. Temelde, bir mutex, bir kaynağa aynı anda yalnızca bir iş parçacığının erişebilmesini garanti eder. Böylece yarış durumlarını (race conditions) ve veri tutarsızlıklarını önler.

Mutex'in Temel Kavramları:

1. Paylaşılan Kaynaklar:

- Birden fazla iş parçacığının aynı kaynağa (değişken, dosya, bellek, vb.) erişmeye çalışması durumunda, her bir iş parçacığının bu kaynağa uygun şekilde erişmesi ve işlem yapabilmesi gerekir.

2. Kilitleme ve Kilit Açma (Lock/Unlock):

- **Lock (Kilitleme):** Bir iş parçacığı, bir kaynağa erişmek için mutex'i "kilitler". Bu, diğer iş parçacıklarının aynı kaynağa erişmesini engeller.

- **Unlock (Kilit Açma):** İşlem tamamlandıktan sonra, iş parçacığı mutex'i "kilit açar", böylece diğer iş parçacıkları kaynağa erişebilir.

3. Yarış Durumu (Race Condition):

- Aynı kaynağa birden fazla iş parçacığı tarafından aynı anda erişim sağlanması, veri tutarsızlıklarına neden olabilir. Mutex, bu tür durumları engellemek için kullanılır.

4. Erişim Kontrolü:

- Mutex, yalnızca bir iş parçacığının kaynağa erişmesine izin verir. Eğer bir iş parçacığı mutex'i kilitlerse, diğer iş parçacıkları beklemek zorunda kalır. Bu, kaynak üzerinde düzenli ve güvenli bir erişim sağlar.

Mutex Nasıl Çalışır?

1. Bir iş parçacığı, bir kaynağa erişmek için mutex'i kilitler.
2. Diğer iş parçacıkları mutex'i kilitlemeye çalıştığında, onlar sırayla bekler (yani, mutex'e "sıra bekleme" davranışı ekler).
3. İş parçacığı kaynağa erişim sağladıktan sonra işlemi bitirir ve mutex'i açar (unlock eder).
4. Mutex açıldığında, bekleyen iş parçacıkları sırayla mutex'i kilitleyerek kaynağa erişir.

Mutex ile İlgili Teorik Konular:

1. Deadlock (Kilitleme Durumu):

- Birden fazla iş parçacığı, birbirlerinin beklediği kaynakları kilitleyerek sonsuz bir bekleme durumuna girebilir. Bu, deadlock olarak bilinir ve bu tür durumları engellemek için uygun kilitleme ve kilit açma düzenlemeleri yapılmalıdır.

2. Priority Inversion (Öncelik Değişimi):

- Düşük öncelikli bir iş parçacığı, yüksek öncelikli bir iş parçacığının beklemesine sebep oluyorsa, bu durum priority inversion olarak adlandırılır. Bu, mutex kullanımı sırasında dikkat edilmesi gereken bir başka önemli durumdur.

❌ Problemler ve Zorluklar

1 Deadlock (Kilitleme):

- Eğer her filozof **aynı anda** sadece sol çatalı alırsa, sağ çatalı alamaz ve **herkes sonsuza kadar bekler**.
- **Tüm filozoflar aç kalır!**

2 Starvation (Aç Kalma):

- Eğer bazı filozoflar sürekli yemek yiyorsa ve bir filozof hiç sıra alamıyorsa, **o filozof sonsuza kadar aç kalabilir**.

3 Concurrency (Eşzamanlılık):

- Filozofların aynı anda hareket ettiği bir sistemde **çatalların paylaşımı nasıl yönetilecek?**

Mutex ve MultiThreading İşlemi

```
#include "philo.h"
```

```
int shared_counter = 0;
```

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void* increment_counter(void* arg)
{
    (void)arg;
    for (int i = 0; i < 1000000; i++) {
        pthread_mutex_lock(&lock); // Mutex kilidini al
        shared_counter++;
        pthread_mutex_unlock(&lock); // Mutex kilidini serbest bırak
    }
    return NULL;
}

int main(int argc, char **argv)
{
    (void)argc;
    (void)argv;
    pthread_t thread1, thread2;

    pthread_create(&thread1, NULL, increment_counter, NULL);
    pthread_create(&thread2, NULL, increment_counter, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Final counter value: %d\n", shared_counter);
    return 0;
}
```

Semaphore Nedir ?

Semaphore, çoklu thread'lerin veya process'lerin paylaşılan kaynaklara erişimini kontrol etmek için kullanılan bir senkronizasyon mekanizmasıdır.

İki türlü semaphore vardır

Binary Semaphore

Binary Semaphore (Mutex gibi çalışır) → 0 veya 1 değeri alır. Bir thread eriştiğinde diğerleri bekler.

Counting Semaphore

Counting Semaphore → Belirli bir sayıya kadar erişime izin verir. **(Bu, Dining Philosophers probleminde kullanılır!)**

Deadlock Durumu ?

Eğer tüm filozoflar aynı anda bir çatal alırsa kimse yemek yiyemez ve sonsuza kadar beklemek zorunda kalırlar.

Thread'den return edilen değeri alma

```
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
#include <time.h>

void* roll_dice() {
    int value = (rand() % 6) + 1;
    int* result = malloc(sizeof(int));
    *result = value;
    // printf("%d\n", value);
    printf("Thread result: %p\n", result);
    return (void*) result;
}

int main(int argc, char* argv[]) {
    int* res;
    srand(time(NULL));
    pthread_t th;
    if (pthread_create(&th, NULL, &roll_dice, NULL) != 0) {
        return 1;
    }
    if (pthread_join(th, (void**) &res) != 0) {
        return 2;
    }
    printf("Main res: %p\n", res);
    printf("Result: %d\n", *res);
    free(res);
    return 0;
}
```

Thread'e parametre geçirme örneği

Hatalı kullanım örneği.

```
int primes[10] = { 2, 3, 5, 7, 11, 13, 17, 19, 23, 29 };

void* routine(void* arg) {
    sleep(1);
    int index = *(int*)arg;
    printf("%d ", primes[index]);
    return (NULL);
}

int main() {
    pthread_t th[10];
    for (int i = 0; i < 10; i++) {
```

```

    if (pthread_create(&th[i], NULL, &routine, &i) != 0) {
        perror("Failed to created thread");
    }
}
for (int i = 0; i < 10; i++) {
    if (pthread_join(th[i], NULL) != 0) {
        perror("Failed to join thread");
    }
}
return (0);
}

```

Sorunun Kaynağı: **i** Değişkeni Aynı Adresi Paylaşıyor

Kodun içinde şu satırda:

```
pthread_create(&th[i], NULL, &routine, &i);
```

Burada **i** değişkeninin adresini (**&i**) tüm thread'lere gönderiyorsun.

Ancak **i** döngü içinde değişmeye devam ediyor! Thread'ler asenkron çalıştığı için, **routine** fonksiyonuna girildiğinde **i**'nin hangi değer olduğu belli olmuyor.

i Değerinin Değişmesi

Şöyle düşünelim:

1. **Ana thread (main) çalışıyor.** **i = 0**, **pthread_create** çağrıldı. Ancak **thread hemen çalışmaya başlamıyor!**
2. **Ana thread i = 1 yapıyor**, ikinci **pthread_create** çağrılıyor.
3. **Ana thread i = 2 yapıyor**, üçüncü **pthread_create** çağrılıyor.
4. ... (bu şekilde **i** büyümeye devam ediyor)
5. **Thread'ler çalışmaya başlıyor**, ama **i zaten 9 olmuş olabilir!**
6. **Tüm thread'ler i = 9 adresini okuyor ve primes[9] olan 29 yazılıyor.**

Yani **thread'ler oluşturulurken** hepsine aynı **&i** adresi verildiği için, **hangi i değerinin okunacağı garanti edilemiyor.**

Doğru kullanım örneği

```

#include "philo.h"

int primes[10] = { 2, 3, 5, 7, 11, 13, 17, 19, 23, 29 };

void* routine(void* arg) {
    sleep(1);
    int index = *(int*)arg;
    printf("%d ", primes[index]);
    free(arg);
    return (NULL);
}

int main() {

```

```
pthread_t th[10];
for (int i = 0; i < 10; i++) {
    int *a = malloc(sizeof(int));
    *a = i;
    if (pthread_create(&th[i], NULL, &routine, a) != 0) {
        perror("Failed to created thread");
    }
}
for (int i = 0; i < 10; i++) {
    if (pthread_join(th[i], NULL) != 0) {
        perror("Failed to join thread");
    }
}
return (0);
}
```

malloc ile Neden Sorun Çözüldü?

Burada `malloc` ile `i` değerinin her thread için ayrı bir kopyasını oluşturduk:

```
int* a = malloc(sizeof(int)); // Yeni bir bellek alanı oluştur
*a = i; // 'i' değerini kopyala
pthread_create(&th[i], NULL, &routine, a);
```

Neden Çalışıyor?

- `malloc` her seferinde **yeni bir bellek alanı** oluşturuyor.
- `a = i` ile o anlık `i` değerini kopyalıyoruz.
- Her thread kendine ait `i` kopyasına sahip oluyor ve global değişikliklerden etkilenmiyor.

Bunu aşağıdaki gibi düşünelim:

Thread ID	Ana <code>i</code> Değeri	malloc ile Ayrılan <code>a</code> Adresi	Thread'in Gördüğü <code>i</code> Değeri
Thread 1	0	0x1000	0
Thread 2	1	0x1010	1
Thread 3	2	0x1020	2
Thread 4	3	0x1030	3

Bu yüzden her thread kendi `i` değerini doğru şekilde okuyor.

Sonuç: `malloc` Kullanmazsak Ne Olur?

Eğer `malloc` kullanmazsan:

- Ana `i` değişkeninin adresini (`&i`) paylaşırsın.
- Ana thread `i` 'yi artırdıkça, thread'ler de değişen değeri görür.
- Sonuç olarak tüm thread'ler yanlış veya beklenmeyen bir `i` değeri kullanabilir.

Eğer `malloc` kullanırsan:

- Her thread için ayrı bir bellek tahsis edilir.

- Her thread kendi bağımsız  kopyasını görür.
- Yarış durumu (race condition) ortadan kalkar ve doğru sonuç elde edilir. 

Summing numbers from an Array

```
#include "philo.h"

int primes[10] = { 2, 3, 5, 7, 11, 13, 17, 19, 23, 29 };

void* routine(void* arg) {
    sleep(1);
    int index = *(int*)arg;
    int sum = 0;
    for (int j = 0; j < 5; j++)
    {
        sum += primes[index + j];
    }
    *(int *)arg = sum;
    return (arg);
}

int main() {
    pthread_t th[2];
    int i;
    for (i = 0; i < 2; i++) {
        int *a = malloc(sizeof(int));
        *a = i * 5;
        if (pthread_create(&th[i], NULL, &routine, a) != 0) {
            perror("Failed to created thread");
        }
    }
    int globalSum = 0;
    for (i = 0; i < 2; i++) {
        int *r;
        if (pthread_join(th[i], (void **) &r) != 0) {
            perror("Failed to join thread");
        }
        globalSum += *r;
        free(r);
    }

    printf("global sum is %d", globalSum);

    return 0;
}
```

Condition Variables in C

(`pthread_cond_signal()` veya `pthread_cond_wait()`) gibi fonksiyonlar External Functs. kısmında yer almıyor !!

Condition Variables (Koşul Değişkenleri), çoklu iş parçacığı (multithreading) programlamasında kullanılan senkronizasyon araçlarıdır. Bu yapılar, bir iş parçacığının belirli bir koşulun gerçekleşmesini beklerken diğer iş parçacığının işlemi tamamlanmasını sağlar. Condition variable'lar, iş parçacıkları arasında iletişimi ve koordinasyonu sağlamaya yönelik önemli bir araçtır.

Görevleri ve Kullanım Amacı:

- **Koşul Bekleme:** Bir iş parçacığı, belirli bir koşulun gerçekleşmesini beklerken bloklanır. Bu, o iş parçacığının ilerleyebilmesi için diğer iş parçacıklarının bir işlemi tamamlanmasını beklediği durumlar için kullanılır. Koşul sağlanana kadar iş parçacığı işlem yapmaz.
- **Koşul Bildirimi:** Diğer bir iş parçacığı, koşulun sağlandığını bildirmek için bir "signal" gönderir. Bu, bekleyen iş parçacığının çalışmasını devam ettirmesine olanak tanır.
- **Senkronizasyon:** Birçok iş parçacığının birbirlerinin durumlarını beklemeleri gereken senaryolarda senkronizasyonu sağlar. Örneğin, bir iş parçacığı bir kaynağı üretirken, diğer bir iş parçacığı bu kaynağı tüketebilir. Bu durumun düzgün bir şekilde çalışabilmesi için condition variable'lar kullanılır.

Nasıl Çalışır:

1. **Mutex ile Birlikte Kullanım:** Condition variable'lar genellikle bir mutex (karşılıklı dışlama) ile birlikte kullanılır. Bu, bir iş parçacığının bir kaynağa erişim için mutex'i kilitlemesi ve condition variable üzerinde beklerken mutex'i serbest bırakması gerektiği anlamına gelir.
 - a. **Bekleme (wait):** Bir iş parçacığı, bir koşulun gerçekleşmesini beklemek için `wait()` fonksiyonunu kullanır. Bu fonksiyon çağrıldığında, iş parçacığı koşulun sağlanması için beklerken, mutex serbest bırakılır. Bu, diğer iş parçacıklarının kaynakları kullanmasına olanak tanır.
2. **Signal Gönderme (signal):** Diğer bir iş parçacığı, koşulun sağlandığını bildirmek için `signal()` veya `broadcast()` fonksiyonlarını kullanır. `signal()`, sadece bir bekleyen iş parçacığını uyarırken, `broadcast()` tüm bekleyen iş parçacıklarını uyarır.
3. **Koşul Gerçekleştiğinde Uyanma:** Koşul sağlandığında, bekleyen iş parçacığı uyanır ve işlemlerine devam eder. Uyanan iş parçacığı, mutex'i yeniden kilitlet ve koşul kontrolü yaparak ilerler.

Özet:

Condition variable'lar, çoklu iş parçacıklı programlarda senkronizasyonu sağlamak için kullanılır. Bir iş parçacığı, belirli bir koşulun gerçekleşmesini beklerken bloklanır ve diğer bir iş parçacığı, koşul sağlandığında onu uyandırır. Bu sayede thread'ler arasında koordinasyon sağlanır ve verimli bir şekilde işlem yapılabilir.

Condition Variable anlamak için gerçek hayat örneği.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>

pthread_mutex_t mutexFuel;
```

```

pthread_cond_t condFuel;
int fuel = 0;

void* fuel_filling(void* arg) {
    for (int i = 0; i < 5; i++) {
        pthread_mutex_lock(&mutexFuel);
        fuel += 15;
        printf("Filled fuel... %d\n", fuel);
        pthread_mutex_unlock(&mutexFuel);
        pthread_cond_signal(&condFuel);
        sleep(1);
    }
}

void* car(void* arg) {
    pthread_mutex_lock(&mutexFuel);
    while (fuel < 40) {
        printf("No fuel. Waiting...\n");
        pthread_cond_wait(&condFuel, &mutexFuel);
        // Equivalent to:
        // pthread_mutex_unlock(&mutexFuel);
        // wait for signal on condFuel
        // pthread_mutex_lock(&mutexFuel);
    }
    fuel -= 40;
    printf("Got fuel. Now left: %d\n", fuel);
    pthread_mutex_unlock(&mutexFuel);
}

int main(int argc, char* argv[]) {
    pthread_t th[2];
    pthread_mutex_init(&mutexFuel, NULL);
    pthread_cond_init(&condFuel, NULL);
    for (int i = 0; i < 2; i++) {
        if (i == 1) {
            if (pthread_create(&th[i], NULL, &fuel_filling, NULL) != 0) {
                perror("Failed to create thread");
            }
        } else {
            if (pthread_create(&th[i], NULL, &car, NULL) != 0) {
                perror("Failed to create thread");
            }
        }
    }

    for (int i = 0; i < 2; i++) {
        if (pthread_join(th[i], NULL) != 0) {
            perror("Failed to join thread");
        }
    }
    pthread_mutex_destroy(&mutexFuel);
}

```

```

    pthread_cond_destroy(&condFuel);
    return 0;
}

```

Signal ile Broadcast Farkı.

```

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>

pthread_mutex_t mutexFuel;
pthread_cond_t condFuel;
int fuel = 0;

void* fuel_filling(void* arg) {
    for (int i = 0; i < 5; i++) {
        pthread_mutex_lock(&mutexFuel);
        fuel += 30;
        printf("Filled fuel... %d\n", fuel);
        pthread_mutex_unlock(&mutexFuel);
        pthread_cond_broadcast(&condFuel);
        sleep(1);
    }
}

void* car(void* arg) {
    pthread_mutex_lock(&mutexFuel);
    while (fuel < 40) {
        printf("No fuel. Waiting...\n");
        pthread_cond_wait(&condFuel, &mutexFuel);
        // Equivalent to:
        // pthread_mutex_unlock(&mutexFuel);
        // wait for signal on condFuel
        // pthread_mutex_lock(&mutexFuel);
    }
    fuel -= 40;
    printf("Got fuel. Now left: %d\n", fuel);
    pthread_mutex_unlock(&mutexFuel);
}

int main(int argc, char* argv[]) {
    pthread_t th[6];
    pthread_mutex_init(&mutexFuel, NULL);
    pthread_cond_init(&condFuel, NULL);
    for (int i = 0; i < 6; i++) {
        if (i == 4 || i == 5) {

```

```

        if (pthread_create(&th[i], NULL, &fuel_filling, NULL) != 0) {
            perror("Failed to create thread");
        }
    } else {
        if (pthread_create(&th[i], NULL, &car, NULL) != 0) {
            perror("Failed to create thread");
        }
    }
}

for (int i = 0; i < 6; i++) {
    if (pthread_join(th[i], NULL) != 0) {
        perror("Failed to join thread");
    }
}
pthread_mutex_destroy(&mutexFuel);
pthread_cond_destroy(&condFuel);
return 0;
}

```

pthread_cond_signal() ve pthread_cond_broadcast() Farkı

1. pthread_cond_signal():

- Sadece **bir** bekleyen iş parçacığını (thread) uyandırır.
- Eğer hiçbir iş parçacığı beklemiyorsa, çağrı boşa gider.

2. pthread_cond_broadcast():

- **Tüm** bekleyen iş parçacıklarını uyandırır.
- Uyandırılan iş parçacıkları mutex'i sırasıyla alır ve çalışmaya devam eder.

Kodda pthread_cond_broadcast() Kullanılmasının Nedeni

Kodda iki farklı thread türü var:

- **fuel_filling()** → Yakıt ekler ve condition variable'ı bildirir.
- **car()** → En az 40 birim yakıt gelmesini bekler.

Burada `pthread_cond_wait()` ile `car()` thread'leri yakıtın dolmasını bekler. Eğer `pthread_cond_signal()` kullanılsaydı, sadece **bir** thread uyanırdı. Ancak **birden fazla araç (car thread'i) aynı anda yakıt beklediği için**, `pthread_cond_broadcast()` kullanılarak **tüm bekleyen araçlara sinyal gönderiliyor**.

Eğer pthread_cond_signal() Kullanılsaydı Ne Olurdu?

- `pthread_cond_signal()` yalnızca **bir** `car()` thread'ini uyandırır.
- Diyelim ki üç araba bekliyor, ancak sadece bir tanesi sinyal alır ve yakıt alır. Geri kalan iki araba **hala beklemeye devam eder**.
- Oysaki yakıt dolduğunda, tüm araçların yeniden kontrol etmesi gerekecek. `broadcast()` bunu sağlıyor.

Bu yüzden, **eğer birden fazla thread'in bekleme ihtimali varsa ve hepsinin durumlarını güncellemesi gerekiyorsa** `pthread_cond_broadcast()` kullanılır.

Kodun çalışma yapısı

`pthread_cond_wait(&condFuel, &mutexFuel);` çağrıldığında, thread **beklemeye geçiyor** ve `condFuel` değişkeni üzerinden bir sinyal (`signal` veya `broadcast`) alana kadar **duruyor**.

Ama **bu bekleme sırasında mutex bırakılıyor!** Böylece başka thread'ler (örneğin `fuel_filling()`) **mutex'i alıp yakıt ekleyebiliyor**.

Sinyal Alındığında (Broadcast veya Signal) Ne Olur?

Bir `fuel_filling()` thread'i `pthread_cond_broadcast(&condFuel);` çağırıldığında:

1. **Bekleyen thread uyanır** (`pthread_cond_wait` sona erer).
2. **Mutex tekrar alınır.**
 - `pthread_cond_wait()` mutex'i serbest bıraktığı için, tekrar çalışmaya devam etmeden önce **mutex'i geri alır**.
 - Bu şu işleme eşdeğerdir:

```
pthread_mutex_unlock(&mutexFuel); // (wait içinde zaten var)
// Burada thread bekler...
pthread_mutex_lock(&mutexFuel); // Sinyal alınca tekrar kilitler
```

3. **Kod, `pthread_cond_wait()` satırından SONRA devam eder.**
 - Yani, `while (fuel < 40)` döngüsüne geri döner.
 - Eğer yakıt **hâlâ 40'tan küçükse**, tekrar `pthread_cond_wait()` çalışır.
 - Eğer yakıt **40 veya daha büyükse**, döngüden çıkar ve şu satır çalışır:

```
fuel -= 40;
printf("Got fuel. Now left: %d\n", fuel);
```

4. **Mutex serbest bırakılır ve thread tamamlanır.**

```
pthread_mutex_unlock(&mutexFuel);
```

- Böylece başka bir `car` thread'i mutex'i alabilir ve devam edebilir.

pthread_t nedir ve Thread'in ID'sini görmek

pthread_t Nedir?

`pthread_t` , POSIX thread'leri (pthread) için kullanılan bir veri türüdür.

Her oluşturulan thread için bir **thread kimliği** (`pthread_t`) döndürülür ve bu kimlik ile thread'leri yönetebiliriz.

pthread_t Nasıl Bir Yapıya Sahip?

`pthread_t` aslında **bir struct veya integer benzeri bir tür olabilir** ve sistemden sisteme değişebilir.

- **Linux'ta** `pthread_t` genellikle **bir `unsigned long int` veya bir struct** olarak tanımlanır.
- **macOS ve BSD tabanlı sistemlerde** `pthread_t` **bir pointer veya struct** olabilir.
- **Windows'ta** POSIX thread'ler (`pthread_t`) kullanılmaz, onun yerine **Windows API'deki thread tanımlamaları (`HANDLE`)** kullanılır.

Örnek Tanımlar (Sistemden Sisteme Farklılık)

Linux (glibc pthread)

Linux sistemlerinde `pthread_t` genellikle şu şekilde tanımlanır:

```
typedef unsigned long int pthread_t
```

Bu, `pthread_t` türünün **bir integer gibi kullanılabileceği** anlamına gelir.

macOS / FreeBSD

Bazı BSD sistemlerinde `pthread_t` bir **struct** olarak tanımlanmıştır:

```
typedef struct {  
    void* __opaque;  
} pthread_t;
```

Bu, `pthread_t` 'nin **pointer benzeri** bir yapı olduğu anlamına gelir.

POSIX Thread ID ve Linux OS Thread ID farkı

```
#include <pthread.h>  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <string.h>  
#include <time.h>  
#include <sys/syscall.h>  
  
#define THREAD_NUM 2  
  
void* routine(void* args) {  
    pthread_t th = pthread_self();  
    printf("%u\n", th);  
    printf("%d\n", (pid_t) syscall(SYS_gettid));  
}  
  
int main(int argc, char *argv[]) {  
    pthread_t th[THREAD_NUM];
```

```

int i;
for (i = 0; i < THREAD_NUM; i++) {
    if (pthread_create(&th[i], NULL, &routine, NULL) != 0) {
        perror("Failed to create thread");
    }
    // printf("%u\n", th[i]);
}

for (i = 0; i < THREAD_NUM; i++) {
    if (pthread_join(th[i], NULL) != 0) {
        perror("Failed to join thread");
    }
}
return 0;
}

```

1. `pthread_t` (Kütüphane Thread Kimliği)

```

pthread_t th = pthread_self();
printf("%u\n", th);

```

Bu, **POSIX Threads (`pthread`)** kütüphanesinin kendi içindeki thread kimliğini döndürür.

- `pthread_t`, sistemden sisteme değişebilir:
 - Linux'ta genellikle bir `unsigned long int` olarak saklanır.
 - Bazı sistemlerde **struct veya pointer olabilir**.
- `pthread_t` değerleri **aynı işlem içindeki thread'ler için benzersizdir**, ancak işlem kimliği (`PID`) ile bağlantılı değildir.

2. `syscall(SYS_gettid)` (Gerçek İşletim Sistemi Thread ID'si)

```

printf("%d\n", (pid_t) syscall(SYS_gettid));

```

Bu çağrı, **Linux çekirdeğinden thread'in gerçek ID'sini alır**.

- Linux'ta **her thread, ayrı bir kernel-level thread (KLT) olarak çalışır**.
- `SYS_gettid` çağrısı ile **Linux çekirdeği tarafından atanan ID** alınır.
- **Bu ID işlem (process) bazında değil, sistem çapında benzersizdir**.

pthread_detach Nedir ?

`pthread_detach(pthread_t thread)` fonksiyonu, bir thread'in ayrıştırılmış (**detached**) modda çalışmasını sağlar.

Bu modda çalışan bir thread, **tamamlandığında kendi kaynaklarını otomatik olarak serbest bırakır** ve `pthread_join` çağrılmasına gerek kalmaz.

Nasıl Çalışır?

Normalde `pthread_create` ile oluşturulan bir thread, **birleştirilebilir (joinable) modda** başlar.

- Eğer `pthread_join(thread, NULL)` çağrılmazsa, **thread'in çıkışıyla ilgili kaynaklar serbest bırakılmaz** ve "zombi thread" durumu oluşur.
- `pthread_detach(thread)` çağrıldığında ise:
 - Thread, **tamamlandığında kendi kaynaklarını otomatik olarak temizler**.
 - `pthread_join` çağtırmaya gerek kalmaz.
 - **Bellek sızıntısı (memory leak) riski ortadan kalkar**.

```
void *routine(void *arg)
{
    (void)arg;
    printf("Routine calisti\n");
    sleep(2);
    printf("Routine bitti\n");
    return (NULL);
}

int main(int argc, char **argv)
{
    (void)argc;
    (void)argv;
    pthread_t th[2];

    for (int i = 0; i < 2; i++)
    {
        if(pthread_create(&th[i], NULL, routine, NULL) != 0)
        {
            printf("Failed to create thread");
            return (1);
        }
        printf("Create calisti\n");

        if(pthread_detach(th[i]) != 0)
        {
            printf("Failed to detach thread\n");
            return (2);
        }
    }

    sleep(3);
    printf("Main bitti \n");

    return (0);
}
```

Join ve Detach farkı.

1. `pthread_join`

- `pthread_join(pthread_t thread, void **retval);` şeklinde kullanılır.
- `pthread_join` çağrılan thread'in tamamlanmasını bekler, yani program akışını durdurur.
- Thread'in dönüş değerini almak için kullanılır.

Örnek Kullanım:

```
pthread_t th;  
pthread_create(&th, NULL, routine, NULL);  
pthread_join(th, NULL); // th thread'inin bitmesini bekler
```

2. `pthread_detach`

- `pthread_detach(pthread_t thread);` şeklinde kullanılır.
- `pthread_detach`, thread'i bağımsız hale getirir. Böylece thread tamamlandığında kendi kaynaklarını serbest bırakır.
- `pthread_detach` çağrılan thread üzerinde artık `pthread_join` kullanılamaz.

Örnek Kullanım:

```
pthread_t th;  
pthread_create(&th, NULL, routine, NULL);  
pthread_detach(th); // th thread'i bağımsız çalışır ve tamamlanınca kendi kendini temizler
```

Ne Zaman Hangisini Kullanmalıyım?

- Eğer bir thread'in bitmesini beklemen gerekiyorsa ve dönüş değerini almak istiyorsan → `pthread_join` kullan.
- Thread'in bitmesini beklemek istemiyorsan ve kendi kendine tamamlanmasını istiyorsan → `pthread_detach` kullan.

Örnek Senaryolar:

- Dosya okuma işlemi yapıyorsan ve sonucu bekleyip işlem yapman gerekiyorsa → `pthread_join` daha mantıklıdır.
- Bir sunucuda, her istemci için yeni bir thread oluşturuyorsan ve istemci bağlantısını takip etmek istemiyorsan → `pthread_detach` daha uygundur.

Eğer `pthread_detach` kullanmazsan ve `pthread_join` da çağırmazsan, thread tamamlandığında bellek sızıntısı (`memory leak`) oluşabilir çünkü thread'in kaynakları temizlenmez.

Mutex ve Cond statik initializer kullanma

1. Statik Initializer (`PTHREAD_MUTEX_INITIALIZER`) Kullanımı

- Mutex veya Condition Variable doğrudan tanımlandığında başlatılmış olur.
- Manuel olarak `pthread_mutex_init()` çağırmaya gerek kalmaz.

- **Global veya statik değişkenlerde kullanılabilir ama dinamik olarak oluşturulan (malloc ile ayrılan) değişkenlerde kullanılamaz.**

Örnek:

```
pthread_mutex_t mutexFuel = PTHREAD_MUTEX_INITIALIZER;  
pthread_cond_t condFuel = PTHREAD_COND_INITIALIZER;
```

Burada **mutex ve condition variable otomatik başlatılır** ve herhangi bir `init` fonksiyonunu çağırmaya gerek kalmaz.

2. `pthread_mutex_init()` ve `pthread_cond_init()` Kullanımı

Eğer **statik initializer kullanmazsan**, `pthread_mutex_init()` ve `pthread_cond_init()` fonksiyonlarını çağırarak başlatman gerekir.

```
pthread_mutex_t mutexFuel;  
pthread_cond_t condFuel;  
  
int main() {  
    pthread_mutex_init(&mutexFuel, NULL);  
    pthread_cond_init(&condFuel, NULL);  
  
    // Thread işlemleri burada yapılır  
  
    pthread_mutex_destroy(&mutexFuel);  
    pthread_cond_destroy(&condFuel);  
    return 0;  
}
```

Peki, Hangi Durumda Hangisini Kullanmalıyım?

1. **Eğer mutex veya condition variable statik (global veya file-scope değişken) ise, `PTHREAD_MUTEX_INITIALIZER` kullanabilirsin.**
 - Daha sade bir kod olur, `init()` çağırmaya gerek kalmaz.
2. **Eğer mutex veya condition variable bir heap'te (`malloc()` ile) oluşturuluyorsa, `pthread_mutex_init()` kullanmalısın.**
 - Statik initializer yalnızca **statik veya global değişkenler** için çalışır.
 - Heap'te oluşturulan değişkenler için **manuel `init()` çağrılmalıdır**.
3. **Destroy (`pthread_mutex_destroy()`) çağırmak gerekli mi?**
 - **Statik initializer kullansan bile, `pthread_mutex_destroy()` çağırmak iyi bir pratiktir.**
 - **Dinamik olarak başlattığın (`pthread_mutex_init()`) nesnelerde `destroy()` kesinlikle çağrılmalıdır**, aksi halde bellek sızıntısı olabilir.

Sonuç

Senin örneğinde `PTHREAD_MUTEX_INITIALIZER` kullanmak mantıklı, çünkü **global değişkenleri başlatıyorsun ve ekstra `init()` çağırmaya gerek yok.**

Eğer değişkenleri **heap'te (`malloc`) oluşturacak olsaydın**, şu şekilde kullanmalıydın:

```
pthread_mutex_t *mutexFuel = malloc(sizeof(pthread_mutex_t));  
pthread_mutex_init(mutexFuel, NULL);
```

```
// Kullanımdan sonra:
pthread_mutex_destroy(&mutexFuel);
free(mutexFuel);
```

Bu senaryoda **statik initializer kullanılamazdı**, çünkü `PTHREAD_MUTEX_INITIALIZER` yalnızca **global veya statik değişkenler** için geçerlidir.

PTHREAD_MUTEX_INITIALIZER ve pthread_mutex_init() FARKI

Statik initializer (`PTHREAD_MUTEX_INITIALIZER`) kullanmanın en büyük avantajı, `pthread_mutex_init()` çağdırmaya gerek kalmadan mutex'in otomatik olarak başlatılmasıdır. Arka planda olan şeyleri detaylı bir şekilde açıklayayım.100

Statik Initializer Arka Planda Ne Yapıyor?

Bir **pthread mutex** (veya condition variable) oluşturulduğunda, onun belirli bir başlangıç durumuna sahip olması gerekir. `PTHREAD_MUTEX_INITIALIZER` kullanırsan, bu başlangıç durumu **derleme zamanında (compile-time) ayarlanır** ve mutex **otomatik olarak başlatılmış** olur.

Örneğin:

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

Bu ifade, `pthread_mutex_t` nesnesini **derleme aşamasında** başlatır. Arka planda `PTHREAD_MUTEX_INITIALIZER`, **sabit bir değerle (genellikle sıfır veya özel bir bit deseni) tanımlanmıştır**. Çoğu sistemde bu, **boş (unlock edilmiş) bir mutex durumudur**.

Peki, pthread_mutex_init() ile Ne Farkı Var?

Eğer `pthread_mutex_init()` kullanırsan, mutex'in **çalışma zamanında (runtime)** başlatılması gerekir:

```
pthread_mutex_t mutex;
pthread_mutex_init(&mutex, NULL);
```

Bu durumda:

1. **Bellekte mutex için yer ayrılır (stack veya heap fark etmez).**
2. `pthread_mutex_init()` çağıldığında, sistem çekirdeğine (`libpthread` kütüphanesine) bir sistem çağırısı (syscall) gider.
3. **Mutex'in iç yapısı sıfırlanır ve hazır hale getirilir.**
4. **Eğer özel bir mutex tipi belirtilmişse (`NULL` yerine `pthread_mutexattr_t` verildiğinde), bu özellikler mutex'e uygulanır.**

Bu ekstra işlem, **statik initializer ile tamamen atlanır**.

Ne Gibi Avantajlar Sağlar?

1. Performans Kazancı

- `pthread_mutex_init()` çalışma zamanında ekstra bir işlem yaptığı için biraz daha **yavaş olabilir**.
- `PTHREAD_MUTEX_INITIALIZER`, **sadece bir sabit değer atama işlemi** olduğundan **daha hızlıdır**.

2. Daha Temiz Kod

- `pthread_mutex_init()` çağırmana gerek kalmaz.

- `init` çağrısını unutma ihtimalin ortadan kalkar.

3. Sistem Kaynaklarından Tasarruf

- `pthread_mutex_init()` çağrısı genellikle **çekirdek seviyesinde (kernel) işlem gerektirir**.
- `PTHREAD_MUTEX_INITIALIZER`, sadece **kütüphane seviyesinde** çalıştığından **ekstra sistem çağrısı (syscall) gerektirmez**.

4. Global Değişkenlerde Kullanım Kolaylığı

- Eğer mutex'i **global bir değişken olarak tanımlarsan**, program başladığında zaten başlatılmış olur.

Ne Zaman `pthread_mutex_init()` Kullanmalıyım?

- Eğer **mutex'in özelliklerini değiştirmek istiyorsan** (`pthread_mutexattr_t` ile özel ayarlar yapmak gibi), `pthread_mutex_init()` kullanmalısın.
- Eğer **mutex'i heap'te (`malloc()`) oluşturuyorsan**, `PTHREAD_MUTEX_INITIALIZER` kullanamazsın, `pthread_mutex_init()` kullanmalısın.
- Eğer mutex'i **dinamik olarak başlatıp sonradan yok etmek istiyorsan**, `pthread_mutex_init()` ve `pthread_mutex_destroy()` çağırmalısın.

Deadlock Durumlarını İnceleyelim

Deadlock (ölü kilit) genellikle aşağıdaki 4 koşulun aynı anda gerçekleşmesiyle oluşur. Bu koşullar **Coffman'ın Deadlock Koşulları** olarak bilinir:

1. Karşılıklı Dışlama (Mutual Exclusion)

Bir kaynak aynı anda sadece **bir işlem (thread) tarafından** kullanılabilir.

- Örneğin, **bir mutex kilitlenirse**, başka bir thread onu kullanamaz.

Örnek:

```
pthread_mutex_t mutex;

void* routine(void* args) {
    pthread_mutex_lock(&mutex);
    // Kritik Bölge (Bu mutex sadece bu thread tarafından kullanılabilir)
    pthread_mutex_unlock(&mutex);
}
```

2. Bekleme Koşulu (Hold and Wait)

Bir thread, **önceden aldığı kaynağı bırakmadan** yeni bir kaynak bekler.

- Thread A, **mutex1'i almış ve mutex2'yi bekliyor**.
- Thread B, **mutex2'yi almış ve mutex1'i bekliyor**.

Örnek (Deadlock Riski!):

```
pthread_mutex_lock(&mutexFuel); // İlk mutex alındı
sleep(1);                       // Diğer thread'in çalışmasına fırsat verildi
```

```
pthread_mutex_lock(&mutexWater); // Burada takılıp kalabilir!
```

3. Geri Alamama (No Preemption)

Bir kaynak tahsis edildikten sonra, **başka bir thread onu zorla alamaz**.

- Eğer bir thread **bir mutex'i kilitlediyse**, başka bir thread bu kilidi zorla açamaz.
- Thread, kendi isteğiyle kilidi bırakana kadar diğerleri beklemek zorundadır.

Örnek:

```
pthread_mutex_lock(&mutex); // Mutex kilitlendi
// Eğer bu noktada thread beklenmedik bir şekilde durursa,
// mutex asla serbest kalmaz ve deadlock oluşur.
```

4. Dairesel Bekleme (Circular Wait)

Bir grup thread, **çember şeklinde birbirini bekler**.

- **Thread A** `mutex1` 'i kilitlemiş ve `mutex2` 'yi bekliyor.
- **Thread B** `mutex2` 'yi kilitlemiş ve `mutex1` 'i bekliyor.

Deadlock Oluşan Örnek:

```
// Thread A:
pthread_mutex_lock(&mutex1);
sleep(1);
pthread_mutex_lock(&mutex2); // Burada takılıp kalabilir!

// Thread B:
pthread_mutex_lock(&mutex2);
sleep(1);
pthread_mutex_lock(&mutex1); // Burada takılıp kalabilir!
```

 Burada Thread A ve Thread B birbirini beklediği için işlem ilerleyemez ve program donup kalır!

Deadlock Hangi Durumlarda Oluşur?

1 Birden Fazla Kilit (Mutex) Kullanıldığında

Eğer birden fazla mutex aynı anda kullanılıyorsa **ve kilitleme sıraları rastgele belirlenmişse**, deadlock riski artar.

2 Birden Fazla Thread'in Birbirine Bağımlı Olduğu Durumlarda

Örneğin, bir thread başka bir thread'in tamamlanmasını bekliyorsa, ancak beklenen thread de bir kaynağa erişmek için ilk thread'i bekliyorsa deadlock oluşur.

3 Kilitler (Mutex) Serbest Bırakılmadığında

Eğer bir thread, **aldığı kilidi bırakmadan bir hata nedeniyle çıkarsa**, diğer thread'ler sonsuza kadar bekler.

4 Yanlış Sıra ile Kilit Kullanımı

Deadlock **genellikle kilitlerin yanlış sırayla alınması nedeniyle oluşur**.

Eğer tüm thread'ler kilitleri **her zaman aynı sırayla alırsa**, deadlock oluşmaz.

◆ Deadlock'tan Kaçınma Yöntemleri

1.  **Tüm Thread'lerin Kilitleri Aynı Sırayla Almasını Sağlamak**
 - Önce `mutex1`, sonra `mutex2` alınmalı.
 - Farklı sıralarda alınırsa deadlock oluşabilir.
2.  `pthread_mutex_trylock()` **Kullanmak**
 - Eğer bir kilit alınamazsa, beklemek yerine işlem iptal edilir veya tekrar denenir.
3.  **Timeout Kullanmak**
 - Kilidi almayı belirli bir süre içinde yapamazsa, thread başka bir iş yapmaya devam eder.
4.  **Kaynakları Önceden Tahsis Etmek**
 - Thread'ler, işlemi başlatmadan önce ihtiyacı olan tüm kaynakları almalı.